

Lecture 21: Join Algorithms

Operator Execution in DBMS

Amit Kumar Dhar

November 27, 2025

Operator Execution

Operator Execution in DBMS

- Queries are executed by composing operators.
- Operators implement relational algebra operations (e.g., Selection, Projection, Join).
- Today's focus: **Join** operators.

- We will focus on binary inner equijoins.
- General form: $R \bowtie_{R.A=S.B} S$.
- Generally, for performance, the smaller table is chosen as the left (outer) relation.

Decision on Join Algorithms

- The choice of join algorithm is based on:
 - Output and cost analysis.
 - Memory availability.
 - Existence of indexes.

Output: Early vs. Late Materialization

- **Early Materialization (Data):**

- Full tuples are produced as soon as a match is found.
- **Pros:** Simple to implement.
- **Cons:** Inefficient if output tuples are large and subsequent operators filter many of them.

- **Late Materialization (Record IDs):**

- Initially, only tuple IDs (RIDs) are produced.
- Full tuples are fetched only when necessary.
- **Pros:** Efficient with subsequent filtering operators.
- **Cons:** More complex implementation.

Cost Analysis

- Two tables: R (outer) and S (inner).
- R has M pages and m tuples.
- S has N pages and n tuples.
- **Cost Metric:** Disk I/O (number of page reads/writes).
- We ignore output and network costs for simplicity.

Why not Cross Product + Selection?

- A join can be expressed as: $R \bowtie S = \sigma_{R.A=S.B}(R \times S)$
- **This is not viable because:**
 - The intermediate result of the cross product ($R \times S$) is enormous ($m \times n$ tuples).
 - This would be extremely slow and consume vast amounts of memory and disk space.

Join Algorithms

Join Algorithms Overview

- Nested Loop Join
 - Naive (Tuple-based)
 - Block (Page-based)
 - Index
- Sort Merge Join
- Hash Join
 - Simple
 - Partitioned
 - Hybrid

Naive Nested Loop Join

Algorithm

```
FOR EACH tuple r in R:  
  FOR EACH tuple s in S:  
    IF r.A == s.B:  
      add <r,s> to result
```



Naive Nested Loop Join: Cost

- For each tuple in R, we scan all of S.
- $\text{Cost} = M (\text{scan R}) + (m * N) (\text{for each tuple of R, scan S})$
- **Example:**
 - R has $M=1000$ pages, $m=100,000$ tuples.
 - S has $N=500$ pages, $n=50,000$ tuples.
 - $\text{Cost} = 1000 + (100,000 * 500) = 50,001,000$ I/Os.
 - This is extremely high!

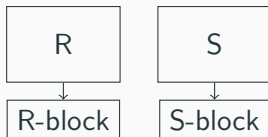
Naive Nested Loop Join: Disadvantages

- Very high cost due to repeated scanning of the inner table S.
- It is tuple-based, which is inefficient. DBMSs operate on pages.
- It doesn't take advantage of available memory.

Block Nested Loop Join

Algorithm

```
FOR EACH block of R:  
  FOR EACH block of S:  
    FOR EACH tuple r in R-block:  
      FOR EACH tuple s in S-block:  
        IF r.A == s.B:  
          add <r,s> to result
```



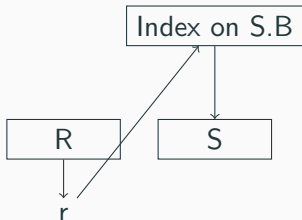
Block Nested Loop Join: Cost

- For each block of R, we scan all blocks of S.
- $\text{Cost} = M (\text{scan } R) + (M * N) (\text{for each block of } R, \text{ scan } S)$
- **Example:**
 - R has $M=1000$ pages.
 - S has $N=500$ pages.
 - $\text{Cost} = 1000 + (1000 * 500) = 501,000 \text{ I/Os.}$
 - A significant improvement over the naive version!

Index Nested Loop Join

Algorithm

```
FOR EACH tuple r in R:  
  use index on S.B to find matching tuples  
  FOR EACH matching tuple s in S:  
    add <r,s> to result
```



Thank You
Questions?